LLM-d installation on OpenShift 4.16

1. context & requirements

context

For the purpose of using AI tools like Claude Code in an air gapped environment, there is a need for an LLM that is large enough to sustain large prompts at high throughput. For this purpose the model https://huggingface.co/moonshotai/Kimi-K2.5 was chosen.

At first there was an attempt for a naive deployment methodology, deploying a classic vLLM image on a kubernetes pod with 8xH100 80GB GPUs; this approach has been fragile under even low loads, so there is a need for an updated approach.

The vLLM error is unclear but one clue that we have is that the pods fail under prompt stress, but not at the vLLM start-up phase nor at the first prompt, therefore it is assumed that it is a resource exhaustion issue.

A new approach for deploying this model is to use a disaggregated model deployment strategy splitting the prefill/decode components of the LLM into different pods, allowing to split the deployment into prefill pods (calculating prompt attention & kv-cache), and decode pods (running the FFN to generate next token). This allows us to maximize utilization on the existing GPUs, which theoretically means that the deployment will be more resilient to resource exhaustion errors.

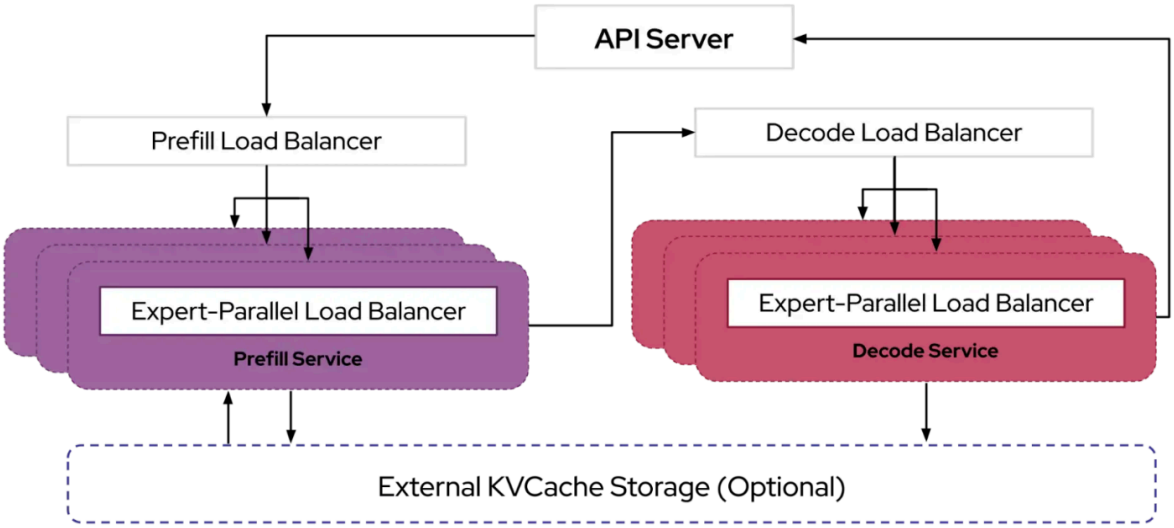

image showing the P/D Disaggregation architecture from llm-d.

Another thing that is critical for implementation here is to set up the inference gateway (IGW) in order to effectively utilize that KVCache.

This is required in order to have a sufficient TTFT for day to day uses in a disaggregated architecture.

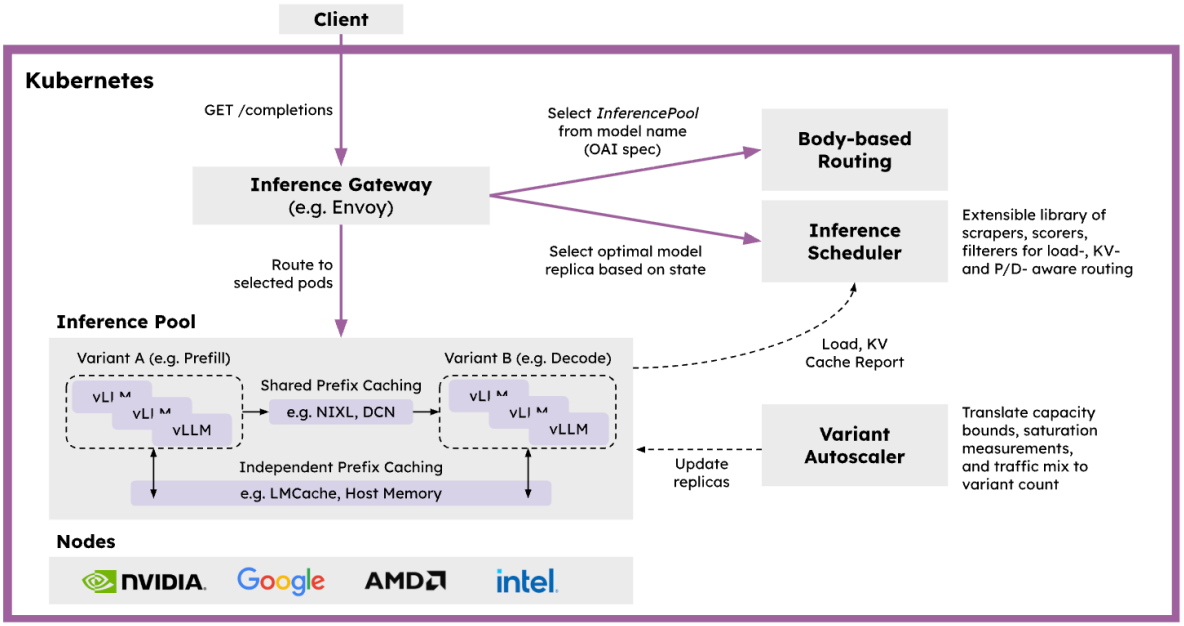

image showing the IGW's architecture from llm-d.

Technical context

The current development environment consists of an OpenShift 4.16 cluster running with NVIDIA's 8xH100 GPUs as worker nodes over basepod & NVLINK, in order to assure high bandwidth and low latency communication between GPUs. This is critical in order for LLM-d to run efficiently.

The cluster runs in an air-gapped environment which means that anything that runs in the internet is inaccessible by default.

2. Installation process

Prerequisites - kGateway

Recommendation - use a DIFFERENT namespace for kGateway and LLM-d

The inference gateway relies on a standalone gateway class operator in order to effectively route traffic in the LLM deployment, for this vLLM supports 3 options:

- GKE Gateway (irrelevant due to not being deployed in GKE)
- Istio gateway (the IT team had trouble installing this)
- kGateway

We chose to use kGateway due to the other options being seemingly more cumbersome.

We installed kGateway v2.1.1 on our OCP cluster.

Required manifests & binaries

- Gateway API v1.2.1 experimental-install.yaml (https://github.com/kubernetes-sigs/gateway-api/releases/tag/v1.2.1)
- kGateway CRDs helm chart v2.1.1 (can be pulled from this helm repo oci://cr.kgateway.dev/kgateway-dev/charts/kgateway-crds)
- kGateway helm chart v2.1.1 (can be pulled from this helm repo oci://cr.kgateway.dev/kgateway-dev/charts/kgateway)
- kGateway controller image (cr.kgateway.dev/kgateway-dev/kgateway:v2.1.1)

Installation steps

- Use kubectl to apply the experimental-install.yaml, in order to extend the kubernetes API to support the necessary gateway resources for HTTPRoute (IGW)

```
kubectl apply -f experiment-install.yaml
```

Validate that the CRDs have been installed

```
kubectl get crds | grep “gateway”
```

- Install the kGateway CRDs helm in your desired namespace

```
helm install -n <YOUR_NAMESPACE> kgateway-crds .
```

- Install the kGateway operator in your desired namespace

  Optional: if the image mirror is not 1-to-1, the image needs in the values under “.image”, and under “.controller.image”

```
helm install -n <YOUR_NAMESPACE> gateway
```

  Validate that the controller pod that was deployed with this helm chart is healthy and that the logs show a successful reconciliation.

Prerequisites - llm-d infrastructure

Before installing LLM-d’s model-service (the LLM itself) we need to set up the proper infrastructure on the cluster.

This includes the LLM-d infra chart and the inferencepool service from kubernetes.

Required manifests & binaries

- LLM-d infra helm chart (helm repo: https://llm-d-incubation.github.io/llm-d-infra/, chart name: llm-d-infra, version: v1.3.6)
- The LLM-d infra common values file for kGateway (https://raw.githubusercontent.com/llm-d/llm-d/refs/heads/main/guides/prereq/gateway-provider/common-configurations/kgateway.yaml)
- The inferencepool helm chart (registry + chart name: oci://registry.k8s.io/gateway-api-inference-extension/charts/inferencepool, version 1.3.1)

- The inferencepool values.yaml from llm-d (v0.5.0) https://raw.githubusercontent.com/llm-d/llm-d/refs/tags/v0.5.0/guides/pd-disaggregation/gaie-pd/values.yaml
- The image of the inference scheduler (ghcr.io/llm-d/llm-d-inference-scheduler:v0.5.0)

Installation steps

- Install the llm-d-infra chart with the values supplied by llm-d

  Optional: if the mirror can not be matched 1-to-1 then the image repo can be defined under ".inferenceExtension.image", which needs to be overridden.

- Validate that the helm chart successfully installed using `helm status` and check that the HTTPRoute that was created is active (using `kubectl get httproute`).

—- END OF PROGRESS 4.3.26 —-

Model service installation

The model service is the helm chart that is responsible for deploying the pods that run the LLM itself.

Required manifests & binaries

- LLM-d modelservice helm chart (helm repo: https://llm-d-incubation.github.io/modelservice/, chart name: modelservice, version: v0.4.7)
- The ms-pd values.yaml file (https://raw.githubusercontent.com/llm-d/llm-d/refs/tags/v0.5.0/guides/pd-disaggregation/ms-pd/values.yaml)
- The llm-d fork of vLLM (ghcr.io/llm-d/llm-d-cuda:v0.5.0)
- The llm-d routing sidecar (ghcr.io/llm-d/llm-d-routing-sidecar:v0.5.0)
- The llm-d pd httproute (for serving) https://raw.githubusercontent.com/llm-d/llm-d/refs/tags/v0.5.0/guides/pd-disaggregation/httproute.yaml

Installation steps

- Create your own values file to override the pd-ms.yaml
    - Change the ".modelArtifacts.uri" to match the kimi-k2.5 huggingface mirror (outside it'd be hf://moonshotai/kimi-k2.5)
    - Change the model size (".modelArtifacts.size") to kimi-k2.5's modelsize (557GB~)
    - Update ".modelArtifacts.authSecretName" to be a secret with the HF TOKEN to huggingface (the secret MUST have a key named HF_TOKEN).
    - Update ".modelArtifacts.name" to "moonshotai/kimi-k2.5"
    - Update the labels appropriately
    - Optional: Change ".routing.sidecar.image" if the mirror is not 1-to-1 to the location of the airgapped image mirror of ghcr.io/llm-d/llm-d-routing-sidecar:v0.5.0
    - Add appropriate pod metadata at ".decode.labels' & ".decode.annotations"
    - Change "Decode.parallelism" to "8" (still untested but I assume its a must due to kimi-k2.5's vram requirements)
    - Change "decode.replicas" to your desired decode replicas (probably 1-2 depending on desired throughput)
    - Optional: Change "decode.containers[0].image" if the mirror is not 1-to-1 to the location of the airgapped image mirror of ghcr.io/llm-d/llm-d-cuda:v0.5.0
    - Optional: Change the "--max-model-len" in "decode.containers[0].args" if it's too low / too high.
    - Remove the rdma/ib: 1 resource from "decode.containers[0].requests" & "decode.containers[0].limits"
    - Add appropriate pod metadata at ".prefill.labels' & ".prefill.annotations"
    - Change "prefill.parallelism" to "1"/"2" (still untested but I assume its low, this requires playing around with)
    - Change "prefill.replicas" to your desired decode replicas (probably 4 depending on desired throughput)
    - Optional: Change "prefill.containers[0].image" if the mirror is not 1-to-1 to the location of the airgapped image mirror of ghcr.io/llm-d/llm-d-cuda:v0.5.0
    - Optional: Change the "--max-model-len" in "prefill.containers[0].args" if it's too low / too high.
    - Remove the rdma/ib: 1 resource from "prefill.containers[0].requests" & "prefill.containers[0].limits"
- Apply the helm chart with the ms-pd values and your overrides
  `helm install -n <YOUR_NAMESPACE> -f values-ms-pd.yaml -f <YOUR_OVERRIDE_VALUES> kimi-k2-5 .`
- Install the HTTPRoute `kubectl apply -f httproute.yaml -n <YOUR_NAMESPACE>`
- Validate the installation

- List the helm charts and make sure their status `helm list -n <YOUR_NAMESPACE>`

  Should look something like:

  ```
  NAME        NAMESPACE   REVISION    UPDATED                                   STATUS
  CHART                       APP VERSION
  gaie-pd     llm-d-pd    1           2025-08-24 12:54:51.231537 -0700 PDT
  deployed    inferencepool-v1.3.0        v1.3.0
  infra-pd    llm-d-pd    1           2025-08-24 12:54:46.983361 -0700 PDT
  deployed    llm-d-infra-v1.3.6          v0.3.0
  ms-pd       llm-d-pd    1           2025-08-24 12:54:56.736873 -0700 PDT
  deployed    llm-d-modelservice-v0.4.5   v0.4.0
  ```

- List all the resources in your namespace

  ```
  kubectl get all -n <YOUR_NAMESPACE>
  ```

  Should look something like that

  ```
  NAME                                          READY   STATUS    RESTARTS   AGE
  pod/gaie-pd-epp-54444ddc66-qv6ds                    1/1     Running   0
  2m35s
  pod/infra-pd-inference-gateway-istio-56d66db57f-zwtzn   1/1     Running   0
  2m41s
  pod/ms-pd-llm-d-modelservice-decode-84bf6d5bdd-jzfjn    2/2     Running   0
  2m30s
  pod/ms-pd-llm-d-modelservice-prefill-86f6fb7cdc-8kfb8   1/1     Running   0
  2m30s
  pod/ms-pd-llm-d-modelservice-prefill-86f6fb7cdc-g6wmp   1/1     Running   0
  2m30s

  NAME                                 TYPE           CLUSTER-IP    EXTERNAL-IP
  PORT(S)                     AGE
  service/gaie-pd-epp                  ClusterIP      10.16.0.255   <none>
  9002/TCP,9090/TCP           2m35s
  service/gaie-pd-ip-bb618139          ClusterIP      None          <none>
  54321/TCP                   2m35s
  service/infra-pd-inference-gateway-istio   ClusterIP      10.16.3.74    10.16.4.3
  15021:31707/TCP,80:34096/TCP   2m41s

  NAME                                          READY   UP-TO-DATE   AVAILABLE   AGE
  deployment.apps/gaie-pd-epp                   1/1     1            1           2m36s
  ```

```
deployment.apps/infra-pd-inference-gateway-istio   1/1     1            1
2m42s
deployment.apps/ms-pd-llm-d-modelservice-decode    1/1     1            1
2m31s
deployment.apps/ms-pd-llm-d-modelservice-prefill   4/4     4            4
2m31s

NAME                                                 DESIRED   CURRENT   READY
AGE
replicaset.apps/gaie-pd-epp-54444ddc66                     1         1         1
2m36s
replicaset.apps/infra-pd-inference-gateway-istio-56d66db57f   1         1         1
2m42s
replicaset.apps/ms-pd-llm-d-modelservice-decode-84bf6d5bdd    1         1         1
2m31s
replicaset.apps/ms-pd-llm-d-modelservice-prefill-86f6fb7cdc   4         4         4
2m31s
```

3. Active maintenance

Post deployment there are some useful tools that could help you assess the performance of models on llm-d and to maintain high SLA.

Monitoring

LLM-d have published multiple grafana dashboards that could be imported into an air-gapped environment in order to assess the deployment's performance (https://llm-d.ai/docs/usage/monitoring#dashboards)

By default LLM-d uses podMonitor to send its metrics to the prometheus operator on the cluster, so metrics will be sent there automatically.

Benchmarking

In order to thoroughly test an LLM deployment's performance, using specialized tools is recommended. Because there are some common pitfalls.

- Inaccurate client simulation - some tools simulate clients by opening a thread for each client and running requests asynchronously.

- Common prefix - using automatically generated responses with a common prefix will cause most of the calculation to be saved in the KV Cache, leading to misleading results in aggregate
- Prompt databases - benchmarking results can heavily skew depending on the prompt databases, different models have been shown to be faster / slower depending on the database, with some models being relatively faster/slower at code tasks and some at summarization and RAG. it is important that you'd use a prompt database that is based on your actual "prompt profile" and not something generic.
- Token calculation - LLM benchmarkers that ask for parameters like "average token count" when generating prompts tend to be less accurate due to tokenizer mismatch, with the benchmarker using its own internal tokenizer and the model using a different tokenizer, leads to a different ITL (inter token latency) calculation and inaccurate results
- Scheduling drift (ADVANCED ISSUE): some benchmarkers have a tendency to not accurately represent their published sending rates or RPS (requests per second), a benchmarker using a high amount of RPS (usually above 10 or 20) will start experiencing drift between the expected time delta between requests and the observed time delta. For example a benchmarker might experience slowdowns at 20 RPS, it'd be expected for a the time delta between requests to be 1000m / 20 = 50ms, but the observed rate would be 60ms, 10ms might not sound like a lot but it reflects an RPS of 1000 / 60 = 16.667 requests per second, which is significantly less, and the higher the RPS the more that the scheduling drift impacts, a drift of 10ms at 100 RPS will cause a HALVING in the rate of requests. **My recommendation** is to use simple benchmarking at low rates and to extrapolate higher rates from that. If you go over 10/20 RPS in a benchmarker use an HTTP proxy like envoy as a man in the middle to assess that performance is in the expected RPS range.

Regarding benchmarkers, no tool is perfect, my recommendation is guideLLM (https://github.com/vllm-project/guidellm) due it being developed in the same project as vLLM, which guarantees a lot of "eyes" on it and a high standard, when issues like the ones listed above come up.

Also, check if huggingface datasets can be mirrored, since this can help a lot when building / using prompt datasets for benchmarking.

Autoscaling and saturation

When measuring models the following mathematical model is usually followed:

- Search for the a models *saturation point* on a given hardware
- Build an autoscaling solution based on when that autoscaling point is reached

The saturation point of a model is the TOKEN THROUGHPUT at which the LLM inference server is fully saturated and can not accept any more requests.

Saturation is measured theoretically by looking at queue size over time, assuming a server is “over-utilized” it will reach after a certain amount of time a point where it cannot naturally process all the requests being thrown at it and the server message queue will start rising.

The ideal token throughput can be found by measuring the token throughput being received once a server is already oversaturated. This rate is the server working at its maximum capacity.

The autoscaling solution built around that would usually involve KEDA (https://keda.sh/) and the vLLM metrics. Where an increase in the measured token throughput will trigger a scale up of the deployment

When to use LLM-d

LLM-d is a complicated and specialized tool for distributed LLM deployments and is intended for large throughput deployments. Before deploying a model on LLM-d please ask yourself the following questions for self assessment:

- How large is this model? Will P/D aggregation make a noticeable improvement to resource utilization?
- What are my latency and throughput KPIs? LLM-d returns latency and throughput improvements on significantly large deployments, on small deployments there might be a latency DOWNGRADE due to the inter-node communication, when compared to the naive vLLM deployment as a classic kubernetes pod.

If you are not sure about the answer to these questions, perform a comparison between the actual results of each method in order to understand which solution is right for you.